

Домашнє завдання 3: Локалізація агента за допомогою сенсора відстані

Вступ до AGI — Практика 3

2026-04-19

Table of contents

1 Вступ	1
1.1 Огляд середовища	2
2 Стартовий код (homework3.py)	2
2.1 Крок 1 — запуск симуляції та збереження стану світу	2
2.2 Крок 2 — збір вимірювань сенсора	2
2.3 Угода про кути	3
3 Завдання	3
3.1 Завдання 1 — Пошук відповідності (Pattern Matching)	3
3.2 Завдання 2 — Порівняння з істинним положенням	4
3.3 Завдання 3 — Аналіз точності	4
3.4 Завдання 4 — Підбір параметрів	5
3.5 Завдання 5 — Обмежений кут огляду	5
4 Форма здачі	6
5 Довідка	6
5.1 Клас World	6
5.2 Клас RangeSensor	7
5.3 Клас _SensorPose	7
5.4 Зв'язок між зсувом індексу та кутом	7

1. Вступ

У цьому завданні ви будете вирішувати задачу **локалізації агента** у двовимірному клітинному автоматі (Гра Життя Конвея). Агент оснащений сенсором відстані, який виконує повний оберт на 360° і вимірює відстань до перешкод (живих клітин) у всіх напрямках.

Ваша мета — реалізувати алгоритм, який за одним виміром сенсора визначає **де знаходиться агент і в якому напрямку він орієнтований**.

1.1. Огляд середовища

Світ — це бінарна сітка розміром $WORLD_W \times WORLD_H$, де 1 — жива клітина (перешкода), 0 — вільний простір. Сітка еволюціонує за правилами Гри Життя Конвея. Після $N_STEPS = 1500$ кроків світ досягає стабільного “дорослого” стану — він зберігається як `ground_truth`.

2. Стартовий код (`homework3.py`)

Стартовий код виконує дві підготовчі дії:

2.1. Крок 1 — запуск симуляції та збереження стану світу

```
world = World(WORLD_W, WORLD_H)
for _ in range(N_STEPS):
    world.step()

ground_truth = world.state.copy()  # shape (H, W), dtype uint8
```

`ground_truth` — це масив NumPy форми $(WORLD_H, WORLD_W)$, де 1 — жива клітина, 0 — порожній простір. Він буде вашою “картою” для порівняння і відновлення.

2.2. Крок 2 — збір вимірювань сенсора

З вільного простору (`ground_truth == 0`) обираються $N_SAMPLES$ випадкових позицій. У кожній позиції сенсор відстані виконує повний оберт на 360° і записує результат.

```
sensor = RangeSensor(
    camera_w = N_RAYS,          # 360 — один промінь на градус
    obs_radius = OBS_RADIUS,    # максимальна дальність (в одиницях сітки)
    ray_step = RAY_STEP,
    fov_half = np.pi,          # ← повний оберт 360°
)
```

Кожен вимір зберігається як об’єкт `RangeMeasurement`:

```
@dataclass
class RangeMeasurement:
    x: float          # позиція сенсора по X
    y: float          # позиція сенсора по Y
    angles_deg: np.ndarray  # кути променів, форма (N_RAYS,)
    ranges: np.ndarray    # виміряні відстані, форма (N_RAYS,)
```

Усі $N_SAMPLES$ вимірів зберігаються у списку `measurements`.

Важлива деталь: усі вимірювання у `measurements` зроблені при орієнтації `orientation = 0` (сенсор “дивиться” на схід). Кут 0° відповідає напрямку вправо (+X), 90° — вниз (+Y), 180° — вліво, 270° — вгору.

2.3. Угода про кути

Масив `angles_deg` будується як:

```
_world_angles_rad = np.linspace(-np.pi, np.pi, N_RAYS)
angles_deg = (_world_angles_rad * 180.0 / np.pi) % 360.0
```

Таким чином, перший промінь (індекс 0) відповідає 180° (захід), середній промінь (індекс `N_RAYS // 2`) — 0° (схід). При обертанні агента на кут θ вимір циклічно зсувається на відповідну кількість індексів.

3. Завдання

3.1. Завдання 1 — Пошук відповідності (Pattern Matching)

Постановка задачі:

Маємо базу даних вимірювань `measurements` — список зі `N_SAMPLES` записів з відомими позиціями та нульовою орієнтацією.

Тестовий агент розміщується у **випадковій вільній клітині** із **випадковою орієнтацією** $\theta_{\text{true}} \in [0^\circ, 360^\circ)$. Сенсор знімає одне вимірювання при цій орієнтації. Позиція та орієнтація агента вам **невідомі** — потрібно їх оцінити.

Алгоритм:

Для кожного збереженого вимірювання m_i (з позицією (x_i, y_i)):

1. Переберіть усі можливі зсуви $\delta \in \{0, 1, \dots, N_RAYS - 1\}$ (циклічний зсув масиву `ranges` відповідає повороту сенсора).
2. Обчисліть відстань між запитом і m_i зі зсувом δ . Наприклад, середньоквадратична відстань:

$$d(m_i, \delta) = \frac{1}{N} \sum_{k=0}^{N-1} (r_{\text{query}}[k] - m_i.\text{ranges}[(k + \delta) \bmod N])^2$$

3. Запам'ятайте найкращий зсув δ_i^* та відповідну відстань d_i^* .

Оцінка позиції та орієнтації:

$$(i^*, \delta^*) = \arg \min_{i, \delta} d(m_i, \delta)$$

$$\hat{x} = x_{i^*}, \quad \hat{y} = y_{i^*}$$

$$\hat{\theta} = \text{angles_deg}[\delta^*] \pmod{360^\circ}$$

Що потрібно реалізувати:

```
def estimate_pose(query_ranges: np.ndarray,
                  measurements: list[RangeMeasurement]) -> tuple:
    """
```

```

Оцінює позицію та орієнтацію агента за одним виміром сенсора.

Args:
    query_ranges: масив форми (N_RAYS,) – вимір тестового агента.
    measurements: база даних вимірювань зі стартового коду.

Returns:
    (x_est, y_est, theta_est_deg) – оцінені позиція та орієнтація в градусах.
"""
# TODO: реалізуйте алгоритм пошуку відповідності
...

```

Підказка щодо ефективності: замість подвійного циклу (по i та по δ) можна використати `np.roll` або перетворення Фур'є для обчислення кореляції. Функція `np.fft.ifft(np.fft.fft(a).conj() * np.fft.fft(b))` обчислює крос-кореляцію за $O(N \log N)$.

3.2. Завдання 2 — Порівняння з істинним положенням

Виберіть `N_TEST = 50` випадкових вільних клітин і випадкових орієнтацій. Для кожної тестової точки:

1. Зніміть вимір сенсора (використовуйте `_SensorPose` зі стартового коду).
2. Викличте `estimate_pose` і отримайте `(x_est, y_est, theta_est)`.
3. Обчисліть помилку позиції та орієнтації:

$$e_{xy} = \sqrt{(x_{\text{est}} - x_{\text{true}})^2 + (y_{\text{est}} - y_{\text{true}})^2}$$

$$e_{\theta} = \min(|\theta_{\text{est}} - \theta_{\text{true}}|, 360^\circ - |\theta_{\text{est}} - \theta_{\text{true}}|)$$

4. Збережіть усі помилки у списки `errors_xy` та `errors_theta`.

Що потрібно реалізувати:

```

def evaluate(measurements, ground_truth, world, N_TEST=50):
    """
    Тестує estimate_pose на N_TEST випадкових позах.

    Returns:
        errors_xy: список помилок позиції (в одиницях сітки)
        errors_theta: список помилок орієнтації (в градусах)
    """
    # TODO
    ...

```

Побудуйте гістограми `errors_xy` та `errors_theta`. Виведіть середнє та медіанне значення помилок.

3.3. Завдання 3 — Аналіз точності

Запустіть `evaluate` і проаналізуйте результати:

- Яка частка тестових поз визначається **точно** (помилка позиції ≤ 1 клітини)?
- Яка типова помилка орієнтації?
- Чи є кореляція між помилкою позиції і помилкою орієнтації?
- Побудуйте scatter plot: вісь X — e_{xy} , вісь Y — e_{θ} .
- Які позиції найважче локалізувати? Візуалізуйте декілька “провальних” випадків (як у стартовому коді — карта + полярний графік).

3.4. Завдання 4 — Підбір параметрів

Дослідіть, як параметри OBS_RADIUS та N_SAMPLES впливають на точність локалізації.

Процедура:

Для кожної комбінації параметрів зі сітки:

```
obs_radii = [5.0, 10.0, 15.0, 20.0]
n_samples_list = [10, 20, 50, 100]
```

1. Перезберіть вимірювання з цими параметрами.
2. Запустіть evaluate і запишіть середню помилку позиції.

Побудуйте теплову карту (plt.imshow або seaborn.heatmap): - Рядки — значення OBS_RADIUS - Стовпці — значення N_SAMPLES - Колір — середня помилка позиції

Питання для усного захисту:

- Яка комбінація параметрів дає найменшу помилку?
- Як змінюється точність зі збільшенням OBS_RADIUS? Чому?
- Як змінюється точність зі збільшенням N_SAMPLES? Чи є межа насичення?
- Який компроміс між OBS_RADIUS і N_SAMPLES є оптимальним?

3.5. Завдання 5 — Обмежений кут огляду

Досі сенсор виконував повний оберт на **360°**. У реальних системах лідар або ультразвуковий сенсор часто мають **обмежений кут огляду** (наприклад, 120° або 180° вперед). Дослідіть, як це впливає на локалізацію.

Ключова ідея — пошук підпатерну:

База даних measurements залишається **незмінною** (повні 360° вимірювання). Тестовий агент тепер знімає лише **частковий вимір** — дугу шириною FOV градусів. Ваш алгоритм має знайти цей підпатерн усередині кожного збереженого повного обертання, ковзаючи вікном відповідної ширини по всіх N_RAYS позиціях.

```
# Ширина вікна у кількості променів для заданого кута огляду:
fov_deg = 120.0
window_size = round(fov_deg / 360.0 * N_RAYS) # кількість суміжних променів

# Пошук найкращого збігу: для кожного зсуву б порівнюємо вікно запиту
# з відповідним фрагментом збереженого вимірювання:
for delta in range(N_RAYS):
    fragment = np.roll(m_i.ranges, -delta)[:window_size]
    cost = np.mean((query_partial - fragment) ** 2)
```

Різниця з Завданням 1: тепер запит `query_partial` має довжину `window_size`, а не `N_RAYS`. Найкращий зсув δ^* дає і позицію (з m_{i^*}), і орієнтацію (кут, що відповідає індексу δ^*).

Процедура:

1. Оберіть декілька значень кута огляду: `fov_deg` $\in$ {360, 180, 120, 60}.
2. Для кожного значення реалізуйте функцію `estimate_pose_partial(query_partial, fov_deg, measurements)`.
3. Запустіть `evaluate` і порівняйте середні помилки позиції та орієнтації.
4. Побудуйте графік: вісь X — кут огляду (°), вісь Y — середня помилка позиції.

Питання для роздумів (підготуйтеся відповісти на захисті):

- При якому куті огляду точність починає суттєво погіршуватись?
- Чому менший кут огляду ускладнює визначення **орієнтації** більше, ніж **позиції**?
- Якщо FOV дуже малий, в базі може знайтись багато однаково схожих фрагментів — що з цим робити?
- Що відбувається в крайньому випадку — дуже вузький промінь (наприклад, 10°)? Чи взагалі можлива однозначна локалізація?

4. Форма задачі

Завдання здається в **усній формі**. Підготуйте код (`.py` або `.ipynb`) і будьте готові продемонструвати та пояснити:

Показати	Що демонструється
<code>estimate_pose</code>	Запустити на прикладі, пояснити алгоритм
Гістограми помилок	Розповісти про середнє, медіану, хвости розподілу
Scatter plot	Пояснити зв'язок між помилками позиції та орієнтації
Теплову карту параметрів	Проаналізувати вплив <code>OBS_RADIUS</code> і <code>N_SAMPLES</code>
Графік FOV vs помилка	Пояснити вплив обмеженого кута огляду

5. Довідка

5.1. Клас World

```
class World:
    width: int          # кількість стовпців
    height: int         # кількість рядків
    state: np.ndarray   # форма (height, width), dtype uint8

    def step(self) -> None: ...          # один крок GoL
    def get_value(self, x, y) -> int: ...
```

5.2. Клас RangeSensor

```
sensor = RangeSensor(  
    camera_w=360,      # кількість променів  
    obs_radius=10.0,   # максимальна дальність  
    ray_step=0.01,     # крок маршування  
    fov_half=np.pi,   #  $\pi \rightarrow$  повний оберт  $360^\circ$   
    noise_std=0.0,  
)  
ranges = sensor.read(pose) #  $\rightarrow$  np.ndarray форми (camera_w,)
```

5.3. Клас _SensorPose

```
pose = _SensorPose(world, x=5.5, y=7.5, orientation=np.radians(45))  
# pose.position      = [5.5, 7.5]  
# pose.orientation = 0.785... # радіани
```

Орієнтація orientation у радіанах: 0 — схід, $\pi/2$ — південь, π — захід, $3\pi/2$ — північ (система координат з Y вниз).

5.4. Зв'язок між зсувом індексу та кутом

```
# angles_deg[k] — кут k-го променя при orientation=0  
# При orientation=theta сенсор зсувається на:  
shift = round(theta / (2 * np.pi) * N_RAYS) % N_RAYS  
# отже: rotated_ranges = np.roll(ranges, shift)
```